High-Performance Request System

Backend Recovery Under Load

Adaptive versus Static Routing

Scenario 5 Benchmark Analysis

Report:	Scenario 5
Focus:	Backend degradation and recovery
Strategies:	RR, LC, WRR, Adaptive V1
Workload:	100 req/s open-loop, 90 s measurement
Repetitions:	Five runs per strategy

October 2026

Abstract

This report examines whether Adaptive V1 restores traffic to Backend 3 after Backend 3 recovers from a sustained latency degradation. The system uses three backends with initial latencies near 20 ms and routing weights $3 : 1 : 2$. At 30 seconds into a 90-second measurement window, Backend 3 is raised to approximately 500 ms; at 60 seconds it is returned to approximately 20 ms. The workload is a 100 request-per-second open-loop stream with 32 workers, five runs per strategy, and 9,000 requests per run.

Round robin (RR) and weighted round robin (WRR) produce median run-level mean latencies of approximately 75.37 ms and 75.47 ms, respectively. Least connections (LC) and Adaptive V1 produce approximately 25.20 ms and 25.24 ms. The static policies maintain their configured Backend 3 share near 33.3% throughout the run, including during degradation. LC and Adaptive reduce Backend 3 to approximately 2% during degradation and restore it to approximately 33% after recovery.

Adaptive's Backend 3 share moves from 33.4% before degradation to 2.0% during steady degradation, 25.0% in the recovery-boundary window, 33.3% in the first complete recovery windows, and 33.3% in sustained recovery. A material routing response is visible in the 60-second boundary window in all five runs. At one-second resolution, full or sustained recovery occurs within one to two seconds. The first post-recovery window still contains slow completions from requests assigned before recovery, so its mean latency and maximum are elevated even though the routing share has already begun to return.

The data support the claim that Adaptive restores traffic to the recovered backend while achieving latency comparable to LC. They do not establish that Adaptive is better than LC. LC also restores Backend 3, so recovery behavior is not unique to Adaptive. The result is a recovery-aware routing demonstration, not a general claim of superiority over dynamic baselines.

1 Objective

The objective of Scenario 5 is to determine whether Adaptive V1 recognizes backend recovery and restores traffic to a previously degraded backend. The scenario is not merely a latency comparison. It tests the full lifecycle:

1. Backend 3 is healthy and receives roughly one-third of assignments.
2. Backend 3 degrades to approximately 500 ms at 30 seconds.
3. Adaptive observes the latency change and moves traffic away.
4. Backend 3 recovers to approximately 20 ms at 60 seconds.
5. Adaptive observes the improvement and makes Backend 3 competitive again.
6. Traffic returns toward Backend 3.

The key question is therefore whether Adaptive restores Backend 3 traffic after recovery, not simply whether its whole-run average latency is low. A credible analysis must also distinguish Adaptive from LC, which is already dynamic and may recover through its in-flight connection signal.

2 Experimental Configuration

Table 1 summarizes the workload. All backends begin near 20 ms. The configured weights are 3 : 1 : 2 for Backend 1, Backend 2, and Backend 3. These are static server weights; Adaptive V1 does not modify them. The degradation and recovery events are scheduled at 30 seconds and 60 seconds within the measurement window.

Table 1: Scenario 5 configuration.

Parameter	Value	Notes
Backends	3	Backend 1, Backend 2, Backend 3
Initial latency	~ 20 ms	All backends before degradation
Weights	3 : 1 : 2	Backend 1 : Backend 2 : Backend 3
Degradation	Backend 3 $\rightarrow\sim 500$ ms	At 30 s into measurement
Recovery	Backend 3 $\rightarrow\sim 20$ ms	At 60 s into measurement
Load	100 req/s	Open-loop request stream
Workers	32	Concurrent request workers
Warm-up	5 s	Excluded from measurement
Measurement	90 s	9,000 requests per run
Repetitions	5 per strategy	20 runs total
Requests	45,000 per strategy	180,000 across all strategies
Errors	0	No failed requests recorded

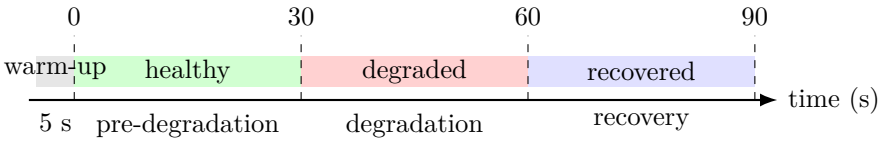


Figure 1: Measurement timeline. The 5-second warm-up is excluded. Backend 3 degrades at 30 seconds and recovers at 60 seconds. The measurement window ends at 90 seconds.

The phase boundaries are derived from the configured `at_s` values and confirmed by the data. The JSON files do not contain an explicit phase field. The analysis uses measurement seconds 0–29 as pre-degradation, 30–59 as degradation, and 60–89 as recovery. The Backend 3 share drops in window 30 and rises in window 60, which confirms those boundaries. Window 60 is a transition window, not a fully recovered steady-state window.

All 20 JSON files are present: five runs for each of four strategies. Every run has 90 windows with indices 0–89, 9,000 requests, and zero errors. Window request counts and per-window backend counts sum to the run total, and the latency count equals the request count. The generated summary file is consistent with the raw files. No run was excluded.

3 Results

3.1 Overall Latency

Table 2 reports the statistical summary. The median and interquartile range are computed across the five per-run mean latencies. P50/P95/P99 are medians of the corresponding per-run percentile values. The pooled mean is weighted by request count; because every run has 9,000 requests, it is also the arithmetic mean of the five run-level means.

Table 2: Overall latency summary. IQR is shown as Q1–Q3.

Strategy	Runs	Req/run	Total	Failed	Median mean	IQR (Q1–Q3)	Pooled mean	Med P50	Med P95	Med P99
RR	5	9,000	45,000	0	75.366	75.364–75.406	75.304	22.2	502.4	502.9
LC	5	9,000	45,000	0	25.203	25.112–25.284	25.199	22.2	22.9	23.5
WRR	5	9,000	45,000	0	75.467	75.452–75.540	75.493	22.3	502.3	502.9
Adaptive	5	9,000	45,000	0	25.236	25.124–25.273	25.157	22.2	22.9	23.5

Table 3: Per-run mean latency in milliseconds.

Run	RR	LC	WRR	Adaptive
Run 1	74.938	25.112	75.428	24.836
Run 2	75.366	25.203	75.540	25.124
Run 3	75.406	25.287	75.467	25.236
Run 4	75.364	25.107	75.452	25.314
Run 5	75.444	25.284	75.580	25.273

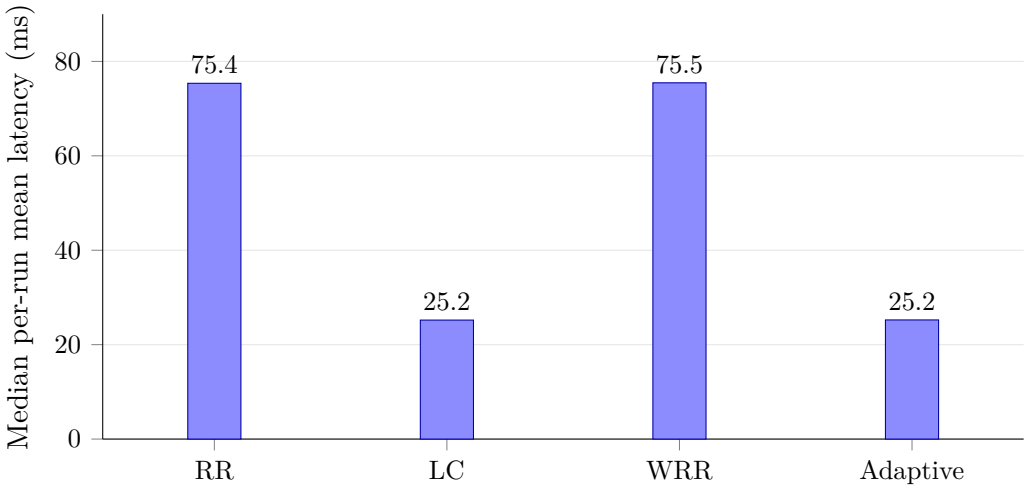


Figure 2: Median per-run mean latency by strategy. Lower is better. The static policies remain near 75 ms; LC and Adaptive remain near 25 ms. The IQRs are given in Table 2.

The strategies divide into two clear groups. RR and WRR remain near 75.3–75.5 ms, while LC and Adaptive remain near 25.2 ms. Adaptive is approximately 66.5% lower than RR and 66.6% lower than WRR on median run-level mean latency. That comparison is against static policies, not against LC.

The Adaptive–LC comparison is deliberately conservative. Adaptive's median is 25.236 ms versus 25.203 ms for LC, which is 0.033 ms higher (about 0.13%). Adaptive's pooled mean is 25.157 ms versus 25.199 ms for LC, which is 0.042 ms lower (about 0.17%). The sign depends on the summary statistic. The difference is small relative to run-to-run variation and is not treated as evidence that either strategy is better.

3.2 Tail Latency

Tail latency separates the static and dynamic strategies more sharply. RR and WRR have median P95/P99 values near 502 ms. LC and Adaptive have median P95 values near 22.9 ms and median P99 values near 23.5 ms. The static policies continue to send a large fraction of requests to Backend 3 while it is slow; the dynamic policies substantially reduce that fraction.

The whole-run P99 of Adaptive is about 23.5 ms even though individual requests reach approximately 503 ms. Only about 0.67% of all Adaptive requests are slow: approximately 2% of one-third of the run. The maximum latency exposes those outliers, while the whole-run P99 does not. During the degradation phase itself, the residual 2% Backend 3 share produces P99 values near 500 ms in many windows.

No request failed in any strategy. Scenario 5 therefore tests latency degradation and recovery, not hard failure, timeout, or retry behavior.

3.3 Adaptive Backend Distribution

Table 4 gives the per-run Adaptive latency distributions and whole-run backend counts. The whole-run Backend 3 share is about 22.7–22.9%, which is the expected mixture of roughly one-third before degradation, approximately 2% during degradation, and roughly one-third after recovery. Backend 1 receives 17,380 requests overall (38.6%), Backend 2 receives 17,372 (38.6%),

and Backend 3 receives 10,248 (22.8%).

Table 4: Per-run Adaptive latency and whole-run backend counts.

Run	Mean	P50	P95	P99	Min	Max	B1	B2	B3
1	24.836	21.8	22.6	23.2	20.38	503.46	3,522	3,431	2,047
2	25.124	22.1	22.9	23.4	20.47	503.32	3,474	3,476	2,050
3	25.236	22.2	22.9	23.5	20.45	504.20	3,460	3,495	2,045
4	25.314	22.2	22.9	23.5	20.44	503.45	3,532	3,420	2,048
5	25.273	22.2	23.0	23.5	20.49	503.68	3,392	3,550	2,058

Table 5 gives the per-run Backend 3 share in each operational phase. The pre-degradation interval is windows 25–29, the final five complete seconds before degradation. The degradation steady interval is windows 35–59, excluding the first five seconds after degradation. The recovery boundary is window 60. The first complete recovery interval is windows 61–64, and sustained recovery is windows 75–89.

Table 5: Per-run Adaptive Backend 3 share by phase, in percent.

Run	Pre 25–29	Deg 35–59	Window 60	Complete 61–64	Sustained 75–89
1	33.00	1.96	23.76	33.25	33.27
2	33.20	1.96	25.74	33.50	33.33
3	33.40	1.96	20.79	33.50	33.20
4	33.60	1.96	20.79	33.25	33.27
5	33.60	1.92	33.66	33.00	33.20

The aggregate Adaptive distribution is 33.4% Backend 3 before degradation, 2.0% during steady degradation, 25.0% in the boundary window, 33.3% in the first complete recovery windows, and 33.3% in sustained recovery. During degradation, the displaced traffic is split almost evenly between Backend 1 and Backend 2: 49.1% and 49.0%. After complete recovery, the distribution returns to approximately 33.4%/33.4%/33.3% for Backend 1/Backend 2/Backend 3.

3.4 Recovery Transition

Figure 3 shows the five-run mean Backend 3 share across the measurement window. The share is stable near one-third before degradation, falls to approximately 2% in window 30, remains near 2% through window 59, rises in window 60, and returns to approximately one-third from window 61 onward.

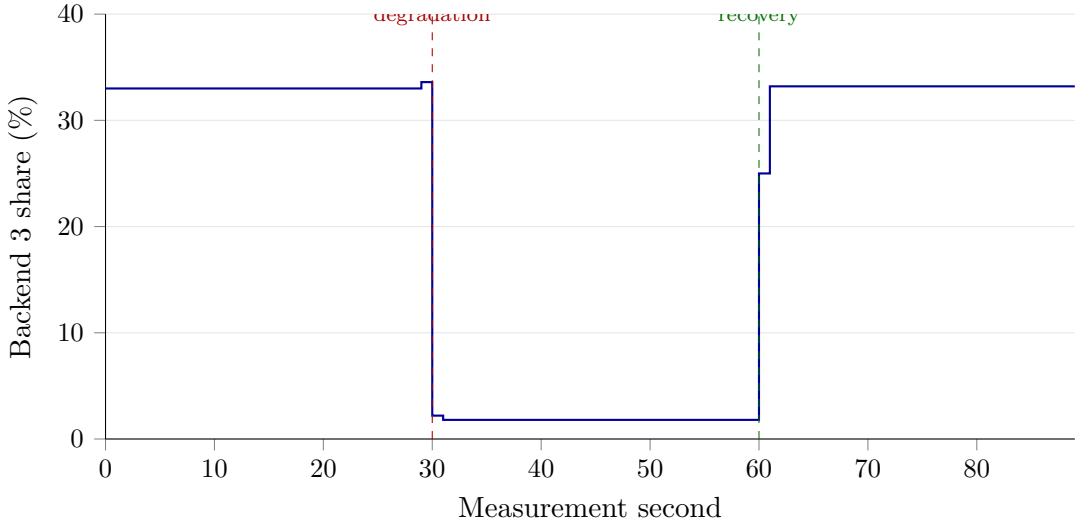


Figure 3: Five-run mean Adaptive Backend 3 share. The plot uses key transition points and holds each value until the next point. Window 30 is the first degraded window; window 60 is the recovery-boundary window.

Table 6 summarizes the boundary windows. The first material routing response is defined as the first post-recovery window with Backend 3 share at least 10%: five times the degraded steady-state share and roughly one-third of the pre-degradation share. Full recovery is defined as the first window within ±3 percentage points of the pre-degradation share. Sustained recovery requires at least three consecutive windows within that band. These thresholds are operational benchmark definitions, not universal ones. They are based on the observed variation: degraded windows vary by about one percentage point, while pre-degradation windows vary by about 0.6 percentage point.

Table 6: Adaptive boundary-window summary across five runs. B3 share and latency are five-run means, with ranges in parentheses.

Second	B3 mean	B3 range	Mean latency	Latency range	Max latency
29 s	33.6%	33.0–34.0%	22.01	21.80–22.25	23.1
30 s	2.2%	2.0–3.0%	30.90	26.92–32.01	503.1
31 s	1.8%	1.0–2.0%	30.75	26.98–31.92	502.3
59 s	1.8%	1.0–2.0%	30.61	27.06–31.89	502.1
60 s	25.0%	20.8–33.7%	26.62	25.57–26.95	502.0
61 s	33.2%	32.0–34.0%	21.82	20.97–22.17	22.8
62 s	33.4%	33.0–34.0%	21.89	20.86–22.23	22.9

All five runs show a material response in window 60. Four runs reach full and sustained recovery in window 61. Run 5 is already within the recovery band in window 60: its Backend 3 share is 33.7% versus a pre-degradation share of 33.6%, and windows 61–62 remain in band, so its full and sustained points are both 60. By window 61, every run is in band. At one-second resolution the data support approximately one second to first response and one to two seconds to full or sustained restoration. They do not support a sub-second claim.

The first post-recovery window has a transition effect. Window 60 has Backend 3 share 20.8–33.7%, but its mean latency is 25.57–26.95 ms, above the sustained level near 22 ms. Its maximum is 500.8–502.7 ms in every run. Requests assigned to Backend 3 shortly before recovery can still

complete at the old 500 ms latency after routing has begun to move back. Routing decisions and request completions are not the same event. By window 61, mean latency is 20.97–22.17 ms and maximum latency is 21.6–23.3 ms. The first post-recovery window is therefore not evidence that Adaptive failed to recover.

The same effect appears at degradation. Window 30 already has Backend 3 share 2.0–3.0%, but its P99 is 501.8–504.2 ms because requests sent before degradation are still completing slowly.

3.5 Comparison with RR, LC, and WRR

Table 7 compares the strategies by phase. RR and WRR keep Backend 3 near 33.3% throughout. Their degradation-phase mean latency is approximately 181.9 ms and 182.1 ms, respectively. Their whole-run median is approximately 75.4–75.5 ms. Their latency falls after recovery only because Backend 3 itself becomes fast again, not because their routing changed.

Table 7: Phase comparison. B3 columns are Backend 3 share; latency columns are mean latency in milliseconds.

Strategy	Pre B3	Deg B3	60 s B3	61–64 B3	Sust B3	Deg mean	60 s mean	61–64 mean	Sust mean
RR	33.2%	33.3%	43.1%	33.2%	33.3%	181.9	92.4	21.9	22.0
LC	33.3%	2.0%	28.9%	33.2%	33.1%	31.6	26.9	22.1	22.0
WRR	33.4%	33.3%	42.2%	33.5%	33.3%	182.1	92.5	22.2	22.1
Adaptive	33.4%	2.0%	25.0%	33.3%	33.3%	31.3	26.6	21.9	22.0

The RR and WRR 60-second values require care. RR shows an apparent Backend 3 share of 43.1% and mean latency 92.4 ms; WRR shows 42.2% and 92.5 ms. These are boundary and catch-up artifacts, not routing recovery. The configured assignments never changed, and window 61 returns to near 33% with mean latency near 22 ms once slow completions have drained.

LC reduces Backend 3 to 2.0% during degradation, restores 28.9% in window 60, and reaches 33.2% in windows 61–64 and 33.1% in sustained recovery. Its degradation-phase mean is 31.6 ms, its whole-run median is 25.203 ms, and its median P95/P99 are 22.9/23.5 ms. During degradation, LC is skewed toward Backend 1: Backend 1 receives about 64.6% and Backend 2 about 33.4%. LC also restores Backend 3, so recovery is not unique to Adaptive.

Adaptive reduces Backend 3 to 2.0% during degradation, restores 25.0% in window 60, and reaches 33.3% in windows 61–64 and 33.3% in sustained recovery. Its degradation-phase mean is 31.3 ms, its whole-run median is 25.236 ms, and its median P95/P99 are 22.9/23.5 ms. During degradation, Adaptive is nearly balanced across Backend 1 and Backend 2: 49.1% and 49.0%. That distribution is more even than LC's, but it does not produce a measurable latency advantage in this scenario.

The residual 2% Backend 3 share explains why LC and Adaptive are not at 22 ms during degradation. With a 500 ms backend receiving 2% of requests, the expected mean is approximately $0.02(500) + 0.98(21) = 30.6$ ms, close to the observed 31.3–31.6 ms. For a static policy that keeps one-third of traffic on Backend 3, the expected degraded mean is approximately $(1/3)(500) + (2/3)(21) = 180.7$ ms, close to the observed 181.9–182.1 ms.

4 Recovery Mechanism

Adaptive V1 combines three normalized signals into a scalar score for each backend:

$$S_i(t) = 0.45\,C_i(t) + 0.40\,R_i(t) + 0.15\,U_i(t), \tag{1}$$

where $C_i(t)$ represents normalized active connections, $R_i(t)$ represents normalized observed response time, and $U_i(t)$ represents normalized CPU utilization. Lower scores are preferred. Backends with lower scores are more likely to be selected on subsequent routing decisions.

The configured server weights remain $3 : 1 : 2$. Adaptive V1 does not modify those weights. It changes the routing preference through the score. In this scenario, response time is the signal that changes materially: Backend 3 rises from about 20 ms to about 500 ms and then returns to about 20 ms. The CPU term is not the manipulated variable. The score is a relative preference, not a hard health check or a circuit breaker.

The observed behavior is consistent with the mechanism. During degradation, Backend 3's response time raises its score relative to the other backends, and its assignment share falls. After recovery, the response-time term falls, Backend 3 becomes competitive again, and its share returns. The data do not include score traces or selection probabilities, so the experiment cannot isolate the contribution of each term or prove the internal causal chain. It can only show that the externally observed routing behavior is consistent with response-time-aware recovery.

5 Comparative Interpretation

The result has two distinct comparisons.

The first is static versus dynamic. RR and WRR keep their configured shares and continue to send approximately one-third of requests to Backend 3 while it is slow. Their whole-run means are near 75 ms and their degraded-phase means are near 181 ms. Adaptive and LC reduce Backend 3 to approximately 2% during degradation and keep whole-run latency near 25 ms. Against RR and WRR, Adaptive's improvement is large and unambiguous.

The second comparison is Adaptive versus LC. LC is already dynamic: a slow backend accumulates in-flight requests, which naturally reduces its share. Adaptive reaches essentially the same latency with a multi-signal score. The median difference is 0.033 ms in LC's favor, while the pooled difference is 0.042 ms in Adaptive's favor. This is not a meaningful separation. The experiment does not establish that Adaptive is better than LC, and it would be incorrect to describe the 66.5% improvement over static policies as an improvement over LC.

The recovery behavior is real but not unique. Both LC and Adaptive restore Backend 3 after it recovers. Adaptive's distinguishing observation in this data is the more even split of displaced traffic across Backend 1 and Backend 2 during degradation: approximately 49.1%/49.0%, compared with LC's approximately 64.6%/33.4%. That difference does not produce a measurable latency benefit here, but it is a genuine distribution difference that may matter in other workloads.

6 Why Static Policies Do Not Restore Routing

Static policies encode a capacity plan, not current health. RR assigns requests evenly and WRR follows the configured $3 : 1 : 2$ weights. Neither rule contains a term for observed latency or

recovery. When Backend 3 degrades, both policies continue to send it roughly one-third of traffic. When Backend 3 recovers, both policies are already sending it one-third of traffic, so there is no routing change to observe.

The latency recovery of RR and WRR is therefore a backend recovery, not a routing recovery. Their post-60-second latency falls because Backend 3 is fast again. Their Backend 3 share is approximately 33.3% in every phase. A static policy can be appropriate when capacity and health are stable, but it cannot reallocate traffic in response to a latency-visible degradation.

7 Why Adaptive Restores Traffic

Adaptive restores traffic because the recovery is visible in the response-time signal. Backend 3's latency returns to approximately 20 ms, its score falls relative to the other backends, and its assignment share rises. In the representative data, the Backend 3 share moves from approximately 2% in window 59 to 25.0% in window 60 and 33.2% in window 61. The first complete recovery windows are at the pre-degradation level, and the sustained recovery interval is also at that level.

The restoration is fast at the available resolution, but it is not instantaneous in the sense of eliminating all slow completions at the boundary. Requests assigned before recovery can complete after the routing decision has changed. This is why window 60 has elevated mean and maximum latency even though Backend 3 share has already risen. By window 61, both the share and the latency distribution are normal.

LC reaches a similar result through in-flight work. Its recovery is an indirect consequence of Backend 3's service time falling; Adaptive's recovery is consistent with a response-time-aware score. The experiment does not prove that the Adaptive score is necessary, only that Adaptive restores traffic in this workload.

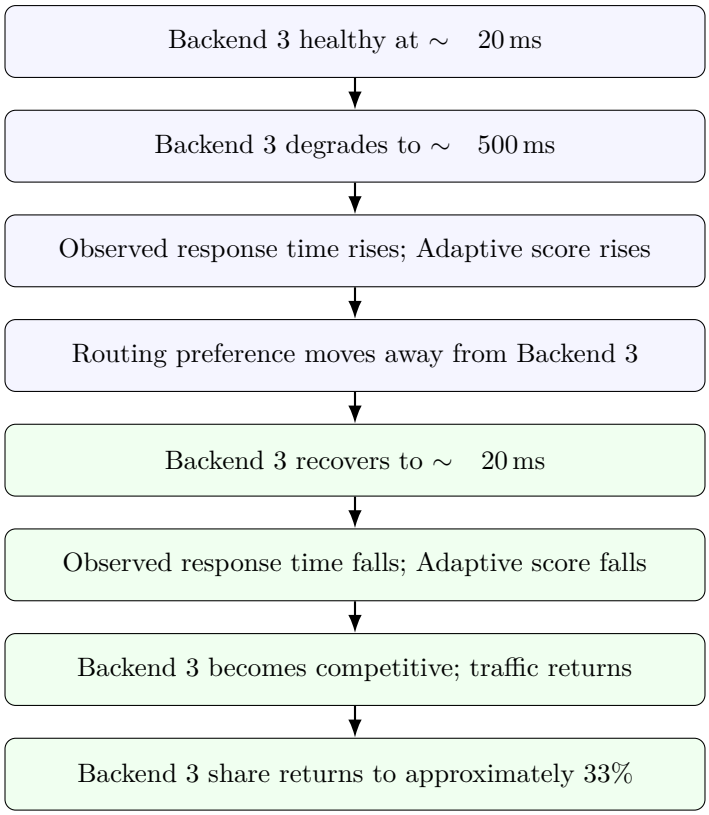


Figure 4: Causal interpretation of the observed Adaptive lifecycle. The experiment shows the external routing behavior; it does not include score traces, so the internal causal chain is an interpretation rather than a directly measured result.

8 Limitations

This scenario has several boundaries that limit generalization.

- **Single recovery schedule.** Backend 3 degrades once at 30 s and recovers once at 60 s. Gradual degradation, partial recovery, repeated flapping, and multiple simultaneous degradations are not tested.
- **One degradation magnitude.** The degraded latency is approximately 500 ms, a large and easily observable change. The results do not establish how Adaptive behaves under smaller or noisier latency changes.
- **One-second windows.** The data cannot resolve sub-second detection, decision, or completion timing. The reported recovery time is an operational, one-second-resolution result.
- **Five runs per strategy.** The data show large static-versus- dynamic differences, but they are not sufficient for a formal significance claim about the small Adaptive-versus-LC difference. No confidence intervals or hypothesis tests were computed.
- **No score traces.** The JSON files do not expose per-request scores or selection probabilities. The experiment cannot isolate the contributions of connections, response time, and CPU.
- **Assignment versus completion.** Per-window backend counts and latencies aggregate completed requests. They do not expose separate assignment and completion timestamps, which is why boundary windows contain transition effects.

- **Open-loop workload.** The generator offers 100 req/s with 32 workers. Overload, queue growth, and client backpressure are not tested.
- **No hard failures.** All requests succeeded. Timeouts, retries, connection resets, health checks, and partial availability are outside this scenario.
- **No production variability.** Wide-area latency variation, packet loss, and heterogeneous clients are not represented.
- **No Adaptive-versus-LC superiority.** The difference is small and changes sign depending on the summary statistic. The data do not support a claim that Adaptive is better than LC.

9 Engineering Interpretation

The first lesson is that static weights are not a recovery mechanism. They encode a capacity plan and remain useful when backend health is stable, but they cannot react when one backend becomes slow or recovers. A weighted policy that continues to send one-third of traffic to a 500 ms backend is behaving as configured, not behaving well.

The second lesson is that response time is a practical signal for this class of recovery. It is observed by the router, it reflects the user-visible effect, and it moves in both directions: up during degradation and down during recovery. A CPU-only signal could miss a latency injection, and a static policy cannot use any signal at all.

The third lesson is that LC is a strong dynamic baseline. It restores Backend 3 without an explicit response-time score. Any claim that Adaptive is superior to LC must demonstrate a workload in which LC's in-flight proxy is insufficient. Scenario 5 does not provide that evidence. It shows comparable latency and a more even distribution of displaced traffic, not a latency win.

The fourth lesson is that recovery control needs damping and observability. A production implementation should expose per-backend shares, score components, and transition behavior. It should include hysteresis or rate limiting to avoid oscillation when a backend flaps, and it should retain a safe residual path for recovery detection. This scenario does not test flapping or hysteresis, but the rapid transition at 60 seconds shows why those mechanisms matter.

Finally, the result should be read as a capability demonstration. Adaptive V1 restores traffic to a recovered backend and is comparable to LC in this workload. It is not a proven general replacement for LC, and it is not evidence that multi-signal routing is always worth its complexity.

10 Conclusion

Scenario 5 shows that Adaptive V1 restores Backend 3 traffic after Backend 3 recovers. The Backend 3 share moves from 33.4% before degradation to 2.0% during steady degradation, 25.0% in the recovery-boundary window, 33.3% in the first complete recovery windows, and 33.3% in sustained recovery. A material response is visible in window 60 in all runs, and full or sustained recovery is present by window 61 at the latest. At one-second resolution, the adaptation time is approximately one to two seconds.

Adaptive's median run-level mean latency is 25.236 ms, comparable to LC at 25.203 ms. RR and WRR remain near 75.4–75.5 ms. Adaptive reduces median whole-run latency by approximately 66.5% relative to RR and 66.6% relative to WRR. These are comparisons against static policies, not against LC.

The static policies never change their Backend 3 share; their latency recovers only because Backend 3 itself recovers. LC also restores Backend 3, so recovery is not unique to Adaptive. The data support the claim that Adaptive achieves performance comparable to LC while restoring traffic to the recovered backend. They do not support the claim that Adaptive is better than LC or that the experiment proves a unique recovery mechanism.

The first post-recovery window contains slow completions from requests assigned before recovery, and should not be interpreted as a recovery failure. The result is a recovery-aware routing demonstration with clear limits: one degradation schedule, one severity, five runs, one-second resolution, and no score traces. The next step is to test multiple degradation and recovery profiles, including flapping and partial recovery, with formal statistical comparison and stability analysis.